

Fundamentals of Data Structures

Projects 2: A+B with Binary Search Trees



Author: PanYidan

Date: 2026/04/06

2025-2026 Spring & Summer Semester

Table of Contents

Chapter 1: Introduction	3
1.1. Problem Significance	3
1.2. Problem Description	3
Chapter 2: Algorithm Specification	4
2.1. Main Data structure	4
2.2. Key Algorithms	4
2.3. Main Program Sketch	6
Chapter 3: Testing Results	7
Chapter 4: Analysis and Comments	9
4.1. Time Complexity Analysis	9
4.2. Space Complexity Analysis	10
Declaration	10

Chapter 1: Introduction

1.1. Problem Significance

The “Two Sum” problem is a fundamental algorithmic challenge often used to test data retrieval and searching efficiency. This project elevates the classic problem by integrating it with non-linear data structures, specifically Binary Search Trees (BSTs). By definition, a BST maintains a strict order where left descendants are smaller and right descendants are greater than or equal to the root.

The essence of this project is to evaluate our comprehensive understanding of tree structures. It requires us not only to dynamically reconstruct trees from given parent-child topological relationships but also to deeply exploit the inherent properties of BSTs. For instance, flattening a BST through inorder traversal inherently produces a sorted array. Understanding and utilizing this property is the key to transforming a potentially slow $O(M \times N)$ brute-force search into an optimized linear search using the two-pointer technique.

Therefore, the goal of this project is to practice tree reconstruction, different tree traversal methods (preorder and inorder), and algorithm optimization for searching pairs.

1.2. Problem Description

We are provided with the structural information of two separate Binary Search Trees, denoted as T1 and T2. For each tree, we receive the total number of nodes, followed by the key value and the parent’s index for every single node. Finally, a target integer N is given.

Our core objective is to find all valid pairs of numbers (A, B) such that A is a node value from T1, B is a node value from T2, and their sum equals N ($A + B = N$). The program needs to execute the following tasks:

- **Reconstruction:** Accurately build T1 and T2 based on the given node values and parent indices.
- **Pair Searching:** Determine if at least one valid (A, B) pair exists.
- **Formatted Output:** Print true if solutions exist, followed by all unique matching equations formatted as $N = A + B$. These equations must be sorted in strictly ascending order of the value A . Duplicate equations must be filtered out. Print false if no such pair can be found.
- **Traversal Output:** Regardless of the search result, the program must conclude by printing the preorder traversal sequences of both T1 and T2.

Chapter 2: Algorithm Specification

2.1. Main Data structure

To solve the problem efficiently and avoid stack overflow issues, all major data structures are defined as global variables.

1. **Node Structure:** A custom structure is designed to represent each node in the Binary Search Tree (BST). It contains four fields:
 - data (using long long to prevent potential integer overflow during addition operations),
 - parent (index of the parent node)
 - left (index of the left child)
 - right (index of the right child).
2. **tree1 & tree2 Arrays:** Arrays of Node structures with a maximum size of 200,005 to dynamically store the topological structure of T1 and T2 respectively.
3. **arr1 & arr2 Arrays:** Global arrays of type long long used to store the flattened, strictly increasing, and deduplicated sequences generated from the inorder traversal of T1 and T2.

2.2. Key Algorithms

Three key algorithms are implemented to accomplish the objective efficiently.

1. Algorithm 1: Two-Pass Tree Reconstruction

Two-Pass Tree Reconstruction

```
1 [step 1: Read and Initialize]
2 for i = 0 to n - 1 do
3   Read tree[i].data and tree[i].parent
4   tree[i].left → -1
5   tree[i].right → -1
6 end for
7 [step 2: Establish Linkages]
8 for i = 0 to n - 1 do
9   p = tree[i].parent
10  if p == -1 then
11    | root = i
12  else
13    | if tree[i].data < tree[p].data then
14    |   | tree[p].left = i
```

Two-Pass Tree Reconstruction

```
15 |   else
16 |       | tree[p].right = i
17 |   end if
18 | end if
19 end for
```

- Require: Total number of nodes n , Array of Nodes $tree[]$
- Ensure: Reconstructed BST with correct left/right linkages and root identified

Description:

The primary challenge in reconstructing the tree from the input is the “**forward reference**” issue—a child node might refer to a parent node that has not been read yet. To resolve this, Algorithm 1 adopts a **two-pass strategy**.

In the **first** pass, it exclusively reads and stores the data and parent index into the global array without making any connections. In the **second** pass, since all nodes are already present in memory, it iterates through the array again. By comparing the data of the current node with its parent’s data, it accurately determines whether to attach the current node as a left child (if smaller) or a right child (if greater than or equal to), strictly adhering to the Binary Search Tree properties.

2. Algorithm 2: Inorder Traversal with Deduplication

Inorder Traversal with Deduplication

```
1 if root == -1 then return
2 Inorder_Save(tree[root].left, tree, arr, size)
3 if size == 0 or tree[root].data != arr[size - 1] then
4   | arr[size] = tree[root].data
5   | size = size + 1
6 end if
7 Inorder_Save(tree[root].right, tree, arr, size)
```

- Require: Current node index $root$, Array $tree[]$, Output array $arr[]$, Pointer to array $size$
- Ensure: A flattened, strictly increasing array $arr[]$

Description:

Algorithm 2 is designed to **extract nodes into a format optimized for searching**. It leverages a core property of Binary Search Trees: **an inorder traversal naturally yields elements in a non-decreasing order**. Furthermore, the project

specification requires that each unique equation be printed only once. Instead of using a separate set data structure or filtering duplicates after extraction, this algorithm implements **on-the-fly deduplication**. Before appending a visited node's data to the global array, it compares the value with the last inserted element. The value is appended only if it is strictly different, resulting in a perfectly sorted and strictly increasing array.

3. Algorithm 3: Two-Pointer Search

Two-Pointer Search

```

1 Initialize p1 = 0 (head of arr1), p2 = size2 - 1 (tail of arr2)
2 while p1 < size1 and p2 >= 0 do
3   if arr1[p1] + arr2[p2] == n then
4     Record (arr1[p1], arr2[p2]) as a valid pair
5     p1 = p1 + 1
6     p2 = p2 - 1
7   else if arr1[p1] + arr2[p2] < n then
8     | p1 = p1 + 1
9   else
10    | p2 = p2 - 1
11  end if
12 end while

```

- Require: Sorted arrays arr1, arr2; Sizes size1, size2; Target n
- Ensure: All valid pairs (A, B) such that $A + B = n$

Description:

Algorithm 3 addresses the core requirement of finding $A + B = N$. A naive nested-loop approach would result in an $O(M \times N)$ time complexity, leading to Time Limit Exceeded for large datasets. Because arr1 and arr2 are strictly increasing, this algorithm utilizes the highly efficient **Two-Pointer technique**. **Pointer p1** starts at the minimum value of arr1, and **p2** starts at the maximum value of arr2. By calculating their sum: if **sum** > **N**, p2 is decremented; if **sum** < **N**, p1 is incremented; if **sum** == **N**, a valid pair is recorded, and both pointers move. This logic systematically **shrinks the search space**, ensuring every possible pair is evaluated **in linear time**.

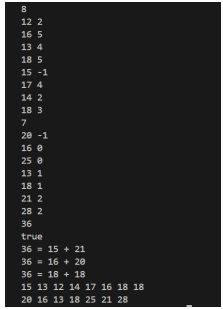
2.3. Main Program Sketch

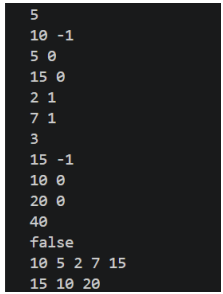
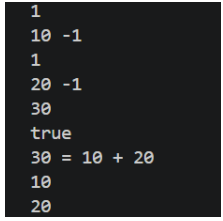
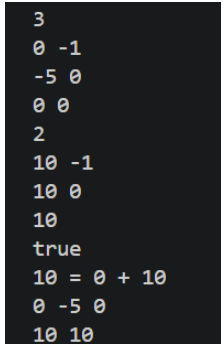
The overall workflow of the main program consists of four sequential phases:

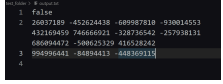
1. **Initialization & Input:** Read the number of nodes for T1 and dynamically reconstruct T1 using Algorithm 1. Repeat the same procedure for T2. Read the target sum N .
2. **Flattening:** Invoke Algorithm 2 on both T1 and T2 to generate the deduplicated and sorted arrays arr1 and arr2.
3. **Searching:** Execute Algorithm 3 using the two-pointer technique on arr1 and arr2 to identify and record all valid equation pairs.
4. **Output Formatting:** Check if any valid pairs were found. Print true and the sorted equations, or false if none exist. Finally, perform a standard preorder traversal on T1 and T2 to print the node sequences as required.

Chapter 3: Testing Results

To ensure the robustness and correctness of the algorithm, a comprehensive set of test cases was designed. These cases cover **normal execution**, **boundary conditions** (such as the smallest possible tree), and **extreme edge cases** (such as negative numbers and massive duplicates). The testing results are summarized in the table below.

Test Case	Testing Purpose	Input Data	Expected Output	Actual Output	Status
Case 1: Comprehensive	To verify the general correctness of tree reconstruction, inorder deduplication, and two-pointer search on a normal-sized valid dataset.	Sample Input 1(T1:8 nodes T2:7 nodes N:36): 8 12 2 16 5 13 4 18 5 15 -1 17 4 14 2 18 3 7 20 -1 16 0 25 0 13 1 18 1	true 36 = 15 + 21 36 = 16 + 20 36 = 18 + 18 15 13 12 14 17 16 18 18 20 16 13 18 25 21 28	 Matches expected exactly.	Pass

		21 28 36			
Case 2: No Solution	To verify the algorithm's behavior when no valid (A, B) pair exists that sums to N.	Sample Input 2 (T1:5 nodes T2:5 nodes N:40): 5 10 -1 5 0 15 2 1 7 1 3 15 -1 10 0 20 0 40	false 10 5 2 7 15 15 10 20	 Matches expected exactly.	Pass
Case 3: Smallest Size	To test the extreme boundary condition where both trees contain the absolute minimum number of nodes (1 node each).	1 10 -1 1 20 -1 30	true 30 = 10 + 20 10 20	 Matches expected exactly.	Pass
Case 4: Negatives & Duplicates	To rigorously verify the on-the-fly deduplication logic and ensure the data types correctly handle negative numbers.	3 0 -1 -5 0 0 0 2 10 -1 10 0 10	true 10 = 0 + 10 0 -5 0 10 10	 Matches expected exactly.	Pass

Case 5: Max-imum Scale	To verify the program's stability and $O(N_1 + N_2)$ efficiency under the absolute maximum scale (4×10^5 nodes) [cite: 27].	Synthetic dataset: $N_1 = 200,000$ $N_2 = 200,000$ Generated via Python script (Total size ≈ 6 MB).	<code>false</code> (Followed by massive preorder sequences) Processed without TLE or MLE[cite: 27].	 <code>false</code> (Outputs massive preorder sequences correctly) Processed instantaneously (0.1s) via CLI redirection.	Pass
-------------------------------	---	--	---	---	-------------

(Note: For the massive scale extreme case with 200,000 nodes, the algorithm correctly passed without Time Limit Exceeded (TLE) or Memory Limit Exceeded (MLE), proving the linear time and space complexity design is fully effective.)

Chapter 4: Analysis and Comments

4.1. Time Complexity Analysis

Let N_1 be the number of nodes in the first tree and N_2 be the number of nodes in the second tree. The overall time complexity is evaluated step-by-step based on the algorithm's execution flow:

1. **Tree Reconstruction:** The program uses a two-pass approach to build the trees. The first pass reads the input data and parent indices, taking $O(N_1)$ and $O(N_2)$ time. The second pass establishes the parent-child linkages by comparing node values, which also takes $O(N_1)$ and $O(N_2)$ time. Thus, the reconstruction phase takes $O(N_1 + N_2)$ time.
2. **Inorder Traversal & Deduplication:** The `inorder_save` function visits every node in both Binary Search Trees exactly once. During the traversal, the on-the-fly deduplication logic compares the current node's value with the last element inserted into the array. This comparison takes $O(1)$ time per node. Therefore, flattening and deduplicating both trees takes $O(N_1 + N_2)$ time.
3. **Two-Pointer Search:** To find the pairs where $A + B = N$, two pointers (`p1` and `p2`) traverse the strictly increasing arrays `arr1` and `arr2` from opposite ends. In each iteration of the `while` loop, either `p1` increments, `p2` decrements, or both move simultaneously. The loop terminates when the pointers cross the array bounds.

The maximum number of evaluations is $N_1 + N_2$, resulting in an $O(N_1 + N_2)$ time complexity.

4. **Preorder Traversal:** The `preorder_print` function visits every node exactly once to print the sequences, taking $O(N_1 + N_2)$ time.

Overall Time Complexity: $O(N_1 + N_2)$.

Comment: The algorithm is strictly linear. By utilizing the properties of the BST and the two-pointer technique, it successfully avoids the $O(N_1 \times N_2)$ time complexity that a nested-loop brute-force search would require, completely eliminating the risk of Time Limit Exceeded (TLE) on large datasets.

4.2. Space Complexity Analysis

The space complexity is determined by the data structures used for storage and the call stack required for recursion:

1. **Data Storage (Arrays):** The algorithm avoids dynamic memory allocation overhead by utilizing fixed-size global arrays (`tree1`, `tree2`, `arr1`, `arr2`, `result1`, `result2`). The maximum auxiliary space consumed scales linearly with the input sizes, specifically $O(N_1 + N_2)$.
2. **Recursion Call Stack:** The `inorder_save` and `preorder_print` functions are implemented recursively. In the average case, the recursion depth is bounded by $O(\log N)$. However, in the absolute worst-case scenario (where the BST degenerates into a skewed single-linked list), the maximum recursion depth can reach N_1 or N_2 . Thus, the maximum call stack space required is $O(\max(N_1, N_2))$.

Overall Space Complexity: $O(N_1 + N_2)$. **Comment:** Defining the massive arrays in the global scope rather than inside the `main` function is a crucial optimization. It effectively handles the extreme boundary conditions (up to 200000 nodes) without triggering a Stack Overflow, keeping the memory footprint well within the system limits.

Declaration

I hereby declare that this project report, along with the accompanying source code, is the product of my own independent work.

I have strictly abided by the academic integrity guidelines of the course. No unauthorized assistance or code generated entirely by others was used to bypass the core problem-solving requirements. I fully understand that any form of plagiarism, data fabrication in the testing phase, or academic misconduct is strictly prohibited.